



XYRO - audit - race

Security Assessment

CertiK Assessed on Feb 19th, 2025





CertiK Assessed on Feb 19th, 2025

XYRO - audit - race

The security assessment was prepared by CertiK, the leader in Web3.0 security.

Executive Summary

TYPES

DeFi

ECOSYSTEM

Ethereum (ETH)

METHODS

Formal Verification, Manual Review, Static Analysis

LANGUAGE

Solidity

TIMELINE

Delivered on 02/19/2025

KEY COMPONENTS

N/A

CODEBASE

[xyro-io](#)[View All in Codebase Page](#)

COMMITTS

[1073bceba6f075d94e32883d4ac38127da2cd811](#)[713050f0bdf4660b07c76e5625eb3350a014978f](#)[d8bc6dcccdd3780cff138f57ad8f32cdf076f99d](#)[View All in Codebase Page](#)

Vulnerability Summary



13

Total Findings

8

Resolved

0

Mitigated

1

Partially Resolved

4

Acknowledged

0

Declined

0 Critical

Critical risks are those that impact the safe functioning of a platform and must be addressed before launch. Users should not invest in any project with outstanding critical risks.

3 Major

2 Resolved, 1 Acknowledged



Major risks can include centralization issues and logical errors. Under specific circumstances, these major risks can lead to loss of funds and/or control of the project.

3 Medium

3 Resolved



Medium risks may not pose a direct risk to users' funds, but they can affect the overall functioning of a platform.

1 Minor

1 Resolved



Minor risks can be any of the above, but on a smaller scale. They generally do not compromise the overall integrity of the project, but they may be less efficient than other solutions.

6 Informational

2 Resolved, 1 Partially Resolved, 3 Acknowledged



Informational errors are often recommendations to improve the style of the code or certain operations to fall within industry best practices. They usually do not affect the overall functioning of the code.

TABLE OF CONTENTS | XYRO - AUDIT - RACE

I **Summary**

[Executive Summary](#)

[Vulnerability Summary](#)

[Codebase](#)

[Audit Scope](#)

[Approach & Methods](#)

I **Review Notes**

[Overview](#)

[External Dependencies](#)

[Addresses](#)

[Privileged Functions](#)

I **Findings**

[RAC-02 : Centralization Risks in Race.sol](#)

[RAC-03 : Incorrect `feedNumbers` Check in `play\(\)`, `playWithPermit\(\)`, and `playWithDeposit\(\)`](#)

[RAC-04 : Not Clearing `depositAmounts` in `finalizeGame\(\)` Will Prevent Players from Participating in Future Games](#)

[CON-01 : `FinalizeGame\(\)` Not Accounting for `topIndex` Pointing to Asset Feed Number index with No Bets](#)

[RAC-05 : No Cap on Fees](#)

[RAC-06 : Should Not `closeGame\(\)` When `finalPricesDiff\[i\]` Equals to `topDiff` in the Loop](#)

[RAC-07 : Missing Check in `setAssetAmount\(\)`](#)

[RAC-01 : Confirmation on Race Game](#)

[RAC-08 : Solidity Version 0.8.23 Won't Work For All Chains Due To MCOPY](#)

[RAC-09 : Event Not Indexed](#)

[RAC-10 : Solidity version 0.8.20 may not work on other chains due to `PUSH0`](#)

[RAC-11 : Missing Check for `minDepositAmount` in `startGame\(\)`](#)

[RAC-12 : Potential Inconsistent Time Range for Price Comparison in Game Finalization](#)

I **Formal Verification**

[Considered Functions And Scope](#)

[Verification Results](#)

I **Appendix**

I **Disclaimer**

CODEBASE | XYRO - AUDIT - RACE

Repository

[xyro-io](#)

Commit

[1073bceba6f075d94e32883d4ac38127da2cd811](#)

[713050f0bdf4660b07c76e5625eb3350a014978f](#)

[d8bc6dcccdd3780cff138f57ad8f32cdf076f99d](#)

[35b382cf4f7f9d12a61871f490716dd4b9b06464](#)

AUDIT SCOPE | XYRO - AUDIT - RACE

2 files audited ● 1 file with Acknowledged findings ● 1 file without findings



ID	Repo	File	SHA256 Checksum
● RAC	xyro-io/smart-contracts	 Race.sol	ea54db8a208ffd01bc5d94bc94f81409b6bf814f05e85e22a3a171ceb01999e4
● RAE	xyro-io/smart-contracts	 Race.sol	c90a81413165d4e6c960dd30df238e737c2769a44ccae2015a01194f03ce1d8a

APPROACH & METHODS | XYRO - AUDIT - RACE

This report has been prepared for XYRO to discover issues and vulnerabilities in the source code of the XYRO - audit - race project as well as any contract dependencies that were not part of an officially recognized library. A comprehensive examination has been performed, utilizing Static Analysis and Manual Review techniques.

The auditing process pays special attention to the following considerations:

- Testing the smart contracts against both common and uncommon attack vectors.
- Assessing the codebase to ensure compliance with current best practices and industry standards.
- Ensuring contract logic meets the specifications and intentions of the client.
- Cross referencing contract structure and implementation against similar smart contracts produced by industry leaders.
- Thorough line-by-line manual review of the entire codebase by industry experts.

The security assessment resulted in findings that ranged from critical to informational. We recommend addressing these findings to ensure a high level of security standards and industry practices. We suggest recommendations that could better serve the project from the security perspective:

- Testing the smart contracts against both common and uncommon attack vectors;
- Enhance general coding practices for better structures of source codes;
- Add enough unit tests to cover the possible use cases;
- Provide more comments per each function for readability, especially contracts that are verified in public;
- Provide more transparency on privileged activities once the protocol is live.

REVIEW NOTES | XYRO - AUDIT - RACE

Overview

The **XYRO** project coordinates a series of smart contracts aimed at various onchain games. The files currently under review include:

- Race.sol

External Dependencies

In **XYRO**, the module inherits or uses a few of the depending injection contracts or addresses to fulfill the need of its business logic. The scope of the audit treats third party entities as black boxes and assume their functional correctness. However, in the real world, third parties can be compromised and this may lead to lost or stolen assets.

Addresses

The following addresses interact at some point with specified contracts, making them an external dependency. All of the following values are initialized either at deployment time or by specific functions in smart contracts.

Race

- `treasury` .

We assume these contracts or addresses are valid and non-vulnerable actors and implementing proper logic to collaborate with the current project.

Also, the following library/contract are considered as the third-party dependencies:

- @openzeppelin/contracts/

Privileged Functions

In the **XYRO** project, the privileged roles are adopted to ensure the dynamic runtime updates of the project, which are specified in the `Centralization` finding.

The advantage of those privileged roles in the codebase is that the client reserves the ability to adjust the protocol according to the runtime required to best serve the community. It is also worth noting the potential drawbacks of these functions, which should be clearly stated through the client's action/plan. Additionally, if the private keys of the privileged accounts are compromised, it could lead to devastating consequences for the project.

To improve the trustworthiness of the project, dynamic runtime updates in the project should be notified to the community.

Any plan to invoke the aforementioned functions should be also considered to move to the execution queue of the

`Timelock` contract.

FINDINGS | XYRO - AUDIT - RACE



13

Total Findings

0

Critical

3

Major

3

Medium

1

Minor

6

Informational

This report has been prepared to discover issues and vulnerabilities for XYRO - audit - race. Through this audit, we have uncovered 13 issues ranging from different severity levels. Utilizing the techniques of Static Analysis & Manual Review to complement rigorous manual code reviews, we discovered the following findings:

ID	Title	Category	Severity	Status
RAC-02	Centralization Risks In Race.Sol	Centralization	Major	● Acknowledged
RAC-03	Incorrect <code>feedNumbers</code> Check In <code>play()</code> , <code>playWithPermit()</code> , And <code>playWithDeposit()</code>	Denial of Service	Major	● Resolved
RAC-04	Not Clearing <code>depositAmounts</code> In <code>finalizeGame()</code> Will Prevent Players From Participating In Future Games	Denial of Service	Major	● Resolved
CON-01	<code>FinalizeGame()</code> Not Accounting For <code>topIndex</code> Pointing To Asset Feed Number Index With No Bets	Design Issue	Medium	● Resolved
RAC-05	No Cap On Fees	Logical Issue	Medium	● Resolved
RAC-06	Should Not <code>closeGame()</code> When <code>finalPricesDiff[i]</code> Equals To <code>topDiff</code> In The Loop	Logical Issue	Medium	● Resolved
RAC-07	Missing Check In <code>setAssetAmount()</code>	Coding Issue	Minor	● Resolved
RAC-01	Confirmation On Race Game	Design Issue	Informational	● Acknowledged
RAC-08	Solidity Version 0.8.23 Won't Work For All Chains Due To MCOPY	Design Issue	Informational	● Resolved

ID	Title	Category	Severity	Status
RAC-09	Event Not Indexed	Design Issue	Informational	● Acknowledged
RAC-10	Solidity Version 0.8.20 May Not Work On Other Chains Due To <code>PUSH0</code>	Logical Issue	Informational	● Acknowledged
RAC-11	Missing Check For <code>minDepositAmount</code> In <code>startGame()</code>	Coding Issue	Informational	● Resolved
RAC-12	Potential Inconsistent Time Range For Price Comparison In Game Finalization	Design Issue	Informational	● Partially Resolved

RAC-02 | CENTRALIZATION RISKS IN RACE.SOL

Category	Severity	Location	Status
Centralization	● Major	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 88, 292, 333, 444, 508, 518, 531, 543	● Acknowledged

Description

In the contract `Race`, the role `DEFAULT_ADMIN_ROLE` has authority over the functions listed below.

- `grantRole()`
- `revokeRole()`
- `setFee()`
- `setAssetAmount()`
- `setTreasury()`
- `setMaxPlayers()`

Any compromise to the `DEFAULT_ADMIN_ROLE` account may allow the hacker to take advantage of this authority and set the fee, adjust game asset amount limits, change the treasury address, and configure the maximum number of players.

In the contract `Race`, the role `GAME_MASTER_ROLE` has authority over the functions listed below.

- `startGame()`
- `setStartingPrice()`
- `finalizeGame()`
- `closeGame()`

Any compromise to the `GAME_MASTER_ROLE` account may allow the hacker to take advantage of this authority and start a new game with specified parameters, set the starting price after `stopPredictAt` time, finalize the game and distribute rewards, and close the current game and refund players.

Recommendation

The risk describes the current project design and potentially makes iterations to improve in the security operation and level of decentralization, which in most cases cannot be resolved entirely at the present stage. We advise the client to carefully manage the privileged account's private key to avoid any potential risks of being hacked. In general, we strongly recommend centralized privileges or roles in the protocol be improved via a decentralized mechanism or smart-contract-based accounts with enhanced security practices, e.g., multisignature wallets. Indicatively, here are some feasible suggestions that would also mitigate the potential risk at a different level in terms of short-term, long-term and permanent:

Short Term:

Timelock and Multi sign (2/3, 3/5) combination *mitigate* by delaying the sensitive operation and avoiding a single point of key management failure.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Assignment of privileged roles to multi-signature wallets to prevent a single point of failure due to the private key compromised;
AND
- A medium/blog link for sharing the timelock contract and multi-signers addresses information with the public audience.

Long Term:

Timelock and DAO, the combination, *mitigate* by applying decentralization and transparency.

- Time-lock with reasonable latency, e.g., 48 hours, for awareness on privileged operations;
AND
- Introduction of a DAO/governance/voting module to increase transparency and user involvement.
AND
- A medium/blog link for sharing the timelock contract, multi-signers addresses, and DAO information with the public audience.

Permanent:

Renouncing the ownership or removing the function can be considered *fully resolved*.

- Renounce the ownership and never claim back the privileged roles.
OR
- Remove the risky functionality.

I Alleviation

[XYRO Team, 02/07/2025]:

We will be using timelock and multisig for this contract as we use for others.

<https://xyro-io.gitbook.io/xyro-whitepaper/smart-contracts>

[CertiK, 02/10/2025]:

We will update the finding status accordingly once the multi-signature and time-lock combination is set. Also, it strongly encourages the project team to periodically revisit the private key security management of all addresses related to centralized roles.

RAC-03 | INCORRECT `feedNumbers` CHECK IN `play()` , `playWithPermit()` , AND `playWithDeposit()`

Category	Severity	Location	Status
Denial of Service	● Major	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 147, 194, 245	● Resolved

Description

In `play()` , `feedNumbers` refers to the token position in the array of Chainlink DataStreams feed IDs, and it should be consistent with the `feedNumbers` in `startGame()` . However, it incorrectly verifies whether `feedNumbers` is less than or equal to `assetFeedNumber.length` :

Race.sol

```
245     function play(uint256 depositAmount, uint8 feedNumbers) public {
246         require(depositAmount >= minDepositAmount, "Wrong deposit amount");
247         require(feedNumbers <= assetFeedNumber.length, "Wrong asset number");
248         .....
249     }
```

This causes some asset feed numbers to be unable to bet in `play()` . The same issue exists in `playWithDeposit()` and `playWithPermit()` .

Proof of Concept

The test used feed numbers ranging from 10 to 6. However, during play, the incorrect feed number (3) can be used to `play()` , while the correct feed number (6) cannot be used to `play()` .

Here is the Foundry test code, along with the mock ERC20 code used for testing:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.24;

import {Test, console} from "forge-std/Test.sol";
import {Treasury} from "../contracts/Treasury.sol";
import {IDataStreamsVerifier, Race} from "../contracts/Race.sol";
import {MockERC20} from "../MockERC20.sol";
import {XyroToken} from "../contracts/XyroToken.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
contract RaceTest is Test {

    Treasury public treasury;
    Race private race;
    MockERC20 public usdt;
    XyroToken public xyroToken;
    address public dataStreamVerifier =
address(uint160(uint256(keccak256("dataStreamVerifier"))));

    address public deployer = address(uint160(uint256(keccak256("deployer"))));
    address public player = address(uint160(uint256(keccak256("player"))));

    function setUp() public {

        vm.startPrank(deployer);

        usdt = new MockERC20("Tether USD", "USDT", 6, 1000_000_000);

        //Initial funding
        usdt.transfer(player, 1_000 * 10 ** usdt.decimals());

        xyroToken = new XyroToken(1000_000_000 * 10 ** 18);

        xyroToken.transfer(player, 50_000 * 10 ** 18);

        address treasuryImpl = address(new Treasury());

        treasury = Treasury(address(new ERC1967Proxy(treasuryImpl, "")));

        treasury.initialize(address(usdt), address(xyroToken));

        treasury.setUpkeep(dataStreamVerifier);

        race = new Race();
        race.setTreasury(address(treasury));

        race.grantRole(keccak256("GAME_MASTER_ROLE"), deployer);
        treasury.grantRole(keccak256("DISTRIBUTOR_ROLE"), address(race));

        vm.stopPrank();
    }
}
```

```
}

function test_playfalse() public{

    uint8[] memory feedNumbers = new uint8[](5);
    bytes[] memory unverifiedReport1 = new bytes[](5);
    bytes[] memory unverifiedReport2 = new bytes[](5);

    for(uint8 i=0;i<5;i++){
        feedNumbers[i] = uint8(10) - i;//[10,9,8,7,6] feed number
        unverifiedReport1[i] = bytes("mock1");
        unverifiedReport2[i] = bytes("mock2");
        vm.mockCall(dataStreamVerifier,
abi.encodeWithSelector(IDataStreamsVerifier.verifyReportWithTimestamp.selector,unver
ifiedReport1[i],feedNumbers[i]),
        abi.encode(int192(10 ** 18),uint32(block.timestamp)));
        vm.mockCall(dataStreamVerifier,
abi.encodeWithSelector(IDataStreamsVerifier.assetId.selector,feedNumbers[i]),
        abi.encode(feedNumbers[i]));
    }

    uint256 amount = 1 * 10 ** usdt.decimals();

    vm.startPrank(deployer);

    race.startGame(uint32(block.timestamp + 2 minutes), uint32(block.timestamp +
1 minutes), 1 * 10 ** usdt.decimals(), address(usdt),feedNumbers);

    vm.stopPrank();

    vm.warp(block.timestamp + 12);

    vm.startPrank(player);

    usdt.approve(address(treasury), 100 * 10 ** usdt.decimals());

    //player play this game with wrong feed number and success
    race.play(amount,uint8(3));//use 3 feed number

    vm.expectRevert();
    //player play this game with correct feed number and revert
    race.play(amount,uint8(6));//use 6 feed number

    vm.stopPrank();

}
}
```

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.24;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

import {ERC20Permit} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";

contract MockERC20 is ERC20 {

    uint8 _decimals;

    constructor(string memory name_, string memory symbol_, uint256 decimals_, uint256
amount) ERC20(name_, symbol_) {

        _decimals = uint8(decimals_);
        _mint(_msgSender(), amount * 10 ** _decimals);

    }

    function decimals() public view override returns (uint8) {
        return _decimals;
    }
}
```

Result:

```

└─ [0] VM::startPrank(0x2BAe636DC0aAF98aB22F66b0C9224F1EdcCD7CC9)
  |   └─ [Return]
  |   └─ [289] MockERC20::decimals() [staticcall]
  |   |   └─ [Return] 6
  |   └─ [24739] MockERC20::approve(ERC1967Proxy:
[0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5], 100000000 [1e8])
  |   |   └─ emit Approval(owner: 0x2BAe636DC0aAF98aB22F66b0C9224F1EdcCD7CC9, spender:
ERC1967Proxy: [0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5], value: 100000000 [1e8])
  |   |   └─ [Return] true
  |   └─ [256109] Race.0.8.24::play(1000000 [1e6], 3)
  |   |   └─ [111586] ERC1967Proxy::fallback(1000000 [1e6],
0x2BAe636DC0aAF98aB22F66b0C9224F1EdcCD7CC9,
0xc71e1de9ad203f5c02d929e66d11798cbc3f04d3ce2e7e45c70bbbd97f60255b, 0)
  |   |   |   └─ [111178] Treasury.0.8.24::depositAndLock(1000000 [1e6],
0x2BAe636DC0aAF98aB22F66b0C9224F1EdcCD7CC9,
0xc71e1de9ad203f5c02d929e66d11798cbc3f04d3ce2e7e45c70bbbd97f60255b, 0)
  |   |   |   [delegatecall]
  |   |   |   |   └─ [2562] MockERC20::balanceOf(ERC1967Proxy:
[0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5]) [staticcall]
  |   |   |   |   |   └─ [Return] 0
  |   |   |   |   └─ [28862]
  |   |   |   MockERC20::transferFrom(0x2BAe636DC0aAF98aB22F66b0C9224F1EdcCD7CC9, ERC1967Proxy:
[0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5], 1000000 [1e6])
  |   |   |   |   └─ emit Transfer(from:
0x2BAe636DC0aAF98aB22F66b0C9224F1EdcCD7CC9, to: ERC1967Proxy:
[0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5], value: 1000000 [1e6])
  |   |   |   |   |   └─ [Return] true
  |   |   |   |   └─ [562] MockERC20::balanceOf(ERC1967Proxy:
[0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5]) [staticcall]
  |   |   |   |   |   └─ [Return] 1000000 [1e6]
  |   |   |   |   └─ [2563]
  |   |   |   XyroToken::balanceOf(0x2BAe636DC0aAF98aB22F66b0C9224F1EdcCD7CC9) [staticcall]
  |   |   |   |   |   └─ [Return] 50000000000000000000000000000000 [5e22]
  |   |   |   |   └─ [266] XyroToken::decimals() [staticcall]
  |   |   |   |   |   └─ [Return] 18
  |   |   |   |   └─ [266] XyroToken::decimals() [staticcall]
  |   |   |   |   |   └─ [Return] 18
  |   |   |   |   └─ [266] XyroToken::decimals() [staticcall]
  |   |   |   |   |   └─ [Return] 18
  |   |   |   |   └─ [266] XyroToken::decimals() [staticcall]
  |   |   |   |   |   └─ [Return] 18
  |   |   |   |   └─ [266] XyroToken::decimals() [staticcall]
  |   |   |   |   |   └─ [Return] 18
  |   |   |   |   └─ [266] XyroToken::decimals() [staticcall]
  |   |   |   |   |   └─ [Return] 18
  |   |   |   |   └─ [266] XyroToken::decimals() [staticcall]
  |   |   |   |   |   └─ [Return] 18
  |   |   |   |   └─ [Return] 60000 [6e4]
  |   |   |   |   └─ [Return] 60000 [6e4]
  |   |   |   └─ [938]
  |   ERC1967Proxy::fallback(0xc71e1de9ad203f5c02d929e66d11798cbc3f04d3ce2e7e45c70bbbd97f6
0255b) [staticcall]

```



```
254     require(
255         feedNumberIndex <= assetFeedNumber.length,
256         "Wrong asset number"
257     );
```

Additionally, the `RaceNewPlayer` event has been updated to log `feedNumberIndex`, but the contract emits the feed number instead, which may lead to inconsistency.

[XYRO team, 02/11/2025]:

The project team resolved this issue by implementing the recommendation suggested. Changes have been reflected in the commit hash: <https://github.com/xyro-io/smart-contracts/tree/d8bc6dcccdd3780cff138f57ad8f32cdf076f99d/contracts>

RAC-04 | NOT CLEARING `depositAmounts` IN `finalizeGame()` WILL PREVENT PLAYERS FROM PARTICIPATING IN FUTURE GAMES

Category	Severity	Location	Status
Denial of Service	● Major	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 333~439	● Resolved

Description

In `finalizeGame`, except for calling `closeGame()` when the game is invalid, the `depositAmounts` of all players are not deleted. In this case, `depositAmounts` will remain greater than 0 and will fail the checks in `play()`, `playWithPermit()`, and `playWithDeposit()`:

Race.sol

```
145     function play(uint256 depositAmount, uint8 feedNumbers) public {
146         .....
147         require(
148             depositAmounts[feedNumbers][msg.sender] == 0,
149             "Already participating"
150         );
151         .....
152     }
```

This will prevent players from participating in future games.

Proof of Concept

Here is the Foundry test code, along with the mock ERC20 code used for testing:

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.24;

import {Test, console} from "forge-std/Test.sol";
import {Treasury} from "../contracts/Treasury.sol";
import {IDataStreamsVerifier,Race} from "../contracts/Race.sol";
import {MockERC20} from "../MockERC20.sol";
import {XyroToken} from "../contracts/XyroToken.sol";
import {ERC1967Proxy} from "@openzeppelin/contracts/proxy/ERC1967/ERC1967Proxy.sol";
contract RaceTest is Test {

    Treasury public treasury;
    Race private race;
    MockERC20 public usdt;
    XyroToken public xyroToken;
    address public dataStreamVerifier =
address(uint160(uint256(keccak256("dataStreamVerifier"))));

    address public deployer = address(uint160(uint256(keccak256("deployer"))));
    address public player1 = address(uint160(uint256(keccak256("player1"))));
    address public player2 = address(uint160(uint256(keccak256("player2"))));

    function setUp() public {

        vm.startPrank(deployer);

        usdt = new MockERC20("Tether USD", "USDT", 6, 1000_000_000);

        //Initial funding
        usdt.transfer(player1, 1_000 * 10 ** usdt.decimals());
        usdt.transfer(player2, 1_000 * 10 ** usdt.decimals());

        xyroToken = new XyroToken(1000_000_000 * 10 ** 18);

        xyroToken.transfer(player1, 50_000 * 10 ** 18);
        xyroToken.transfer(player2, 50_000 * 10 ** 18);

        address treasuryImpl = address(new Treasury());

        treasury = Treasury(address(new ERC1967Proxy(treasuryImpl, "")));

        treasury.initialize(address(usdt), address(xyroToken));

        treasury.setUpkeep(dataStreamVerifier);

        race = new Race();
        race.setTreasury(address(treasury));

        race.grantRole(keccak256("GAME_MASTER_ROLE"), deployer);
    }
}
```

```
        treasury.grantRole(keccak256("DISTRIBUTOR_ROLE"), address(race));

        vm.stopPrank();
    }

    function test_playAgainFailed() public{

        uint8[] memory feedNumbers = new uint8[](5);
        bytes[] memory unverifiedReport1 = new bytes[](5);
        bytes[] memory unverifiedReport2 = new bytes[](5);

        for(uint8 i=0;i<5;i++){
            feedNumbers[i] = i;//[0,1,2,3,4] feed number
            unverifiedReport1[i] = bytes("mock1");
            unverifiedReport2[i] = bytes("mock2");
            vm.mockCall(dataStreamVerifier,
abi.encodeWithSelector(IDataStreamsVerifier.verifyReportWithTimestamp.selector, unverifiedReport1[i], feedNumbers[i]),
            abi.encode(int192(10 ** 18), uint32(block.timestamp)));
            vm.mockCall(dataStreamVerifier,
abi.encodeWithSelector(IDataStreamsVerifier.assetId.selector, feedNumbers[i]),
            abi.encode(feedNumbers[i] + 1));
        }

        uint256 amount = 1 * 10 ** usdt.decimals();

        vm.startPrank(deployer);

        race.startGame(uint32(block.timestamp + 2 minutes), uint32(block.timestamp + 1 minutes), 1 * 10 ** usdt.decimals(), address(usdt), feedNumbers);

        vm.stopPrank();

        vm.warp(block.timestamp + 12);

        vm.startPrank(player1);

        usdt.approve(address(treasury), 100 * 10 ** usdt.decimals());

        //player1 play this game with feed number 0
        race.play(amount, uint8(0));

        vm.stopPrank();

        vm.startPrank(player2);

        usdt.approve(address(treasury), 100 * 10 ** usdt.decimals());
```

```
//player2 play this game with wrong feed number 1
race.play(amount,uint8(1));

vm.stopPrank();

//set starting price
vm.warp(block.timestamp + 1 minutes);

for(uint8 i=0;i<5;i++){
    vm.mockCall(dataStreamVerifier,
abi.encodeWithSelector(IDataStreamsVerifier.verifyReportWithTimestamp.selector,unverifiedReport1[i],feedNumbers[i]),
    abi.encode(int192(10 ** 18),uint32(block.timestamp)));
}

vm.startPrank(deployer);

race.setStartingPrice(unverifiedReport1);

vm.stopPrank();

//finalize game,and player1 win
vm.warp(block.timestamp + 1 minutes);

vm.mockCall(dataStreamVerifier,
abi.encodeWithSelector(IDataStreamsVerifier.verifyReportWithTimestamp.selector,unverifiedReport2[0],feedNumbers[0]),
    abi.encode(int192(2 * 10 ** 18),uint32(block.timestamp)));

for(uint8 i=1;i<5;i++){
    vm.mockCall(dataStreamVerifier,
abi.encodeWithSelector(IDataStreamsVerifier.verifyReportWithTimestamp.selector,unverifiedReport2[i],feedNumbers[i]),
    abi.encode(int192(10 ** 18),uint32(block.timestamp)));
}

vm.startPrank(deployer);

race.finalizeGame(unverifiedReport2);

vm.stopPrank();

// now new round start

vm.warp(block.timestamp + 1 minutes);

vm.startPrank(deployer);

race.startGame(uint32(block.timestamp + 2 minutes), uint32(block.timestamp +
1 minutes), 1 * 10 ** usdt.decimals(), address(usdt),feedNumbers);
```

```
vm.stopPrank();

//player1 and player2 can't play game

vm.startPrank(player1);

//player1 play this game with feed number 0 and revert
vm.expectRevert();

race.play(amount,uint8(0));

vm.stopPrank();

vm.startPrank(player2);

usdt.approve(address(treasury), 100 * 10 ** usdt.decimals());

//player1 play this game with feed number 1 and revert
vm.expectRevert();

race.play(amount,uint8(1));

vm.stopPrank();
    }
}
```

```
// SPDX-License-Identifier: UNLICENSED
pragma solidity ^0.8.24;

import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";

import {ERC20Permit} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";

contract MockERC20 is ERC20 {

    uint8 _decimals;

    constructor(string memory name_, string memory symbol_, uint256 decimals_, uint256
amount) ERC20(name_, symbol_) {

        _decimals = uint8(decimals_);
        _mint(_msgSender(), amount * 10 ** _decimals);

    }

    function decimals() public view override returns (uint8) {
        return _decimals;
    }
}
```

Result:


```

└─ [0] VM::startPrank(0x2668bFda9baa2cCC048B99752fdB684789d97233)
  |   └─ [Return]
└─ [0] VM::expectRevert(custom error 0xf4844814)
  |   └─ [Return]
└─ [1064] Race.0.8.24::play(1000000 [1e6], 0)
  |   └─ [Revert] revert: Already participating
└─ [0] VM::stopPrank()
  |   └─ [Return]
└─ [0] VM::startPrank(0x7D7F4146c72cF8F98541A579Bd0755D83e0f0421)
  |   └─ [Return]
└─ [289] MockERC20::decimals() [staticcall]
  |   └─ [Return] 6
└─ [2739] MockERC20::approve(ERC1967Proxy:
[0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5], 100000000 [1e8])
  |   └─ emit Approval(owner: 0x7D7F4146c72cF8F98541A579Bd0755D83e0f0421, spender:
ERC1967Proxy: [0x5E6c05cE12c05b908937bc2D148a0437DaDE4Ba5], value: 100000000 [1e8])
  |   └─ [Return] true
└─ [0] VM::expectRevert(custom error 0xf4844814)
  |   └─ [Return]
└─ [1064] Race.0.8.24::play(1000000 [1e6], 1)
  |   └─ [Revert] revert: Already participating
└─ [0] VM::stopPrank()
  |   └─ [Return]
└─ [Stop]

```

Suite result: ok. 1 passed; 0 failed; 0 skipped; finished in 4.22ms (2.12ms CPU time)

Recommendation

After reward distribution in `finalizeGame`, clear all players' `depositAmounts` similar to how it's done in `closeGame()`.

Alleviation

[XYRO team, 02/09/2025]:

Fixed this issue, please check commit hash: 713050f0bdf4660b07c76e5625eb3350a014978f

[CertiK, 02/10/2025]:

Thank you for your update and response. However, in the fix, `finalizeGame()` first deletes

`assetData[assetFeedNumber[i]]`, and then iterates through the `player` array in `assetData[assetFeedNumber[i]]`:

```
457     for (uint i; i < assetFeedNumber.length; i++) {
458         delete assetData[assetFeedNumber[i]];
459         for (
460             uint k;
461             k < assetData[assetFeedNumber[i]].players.length;
462             k++
463         ) {
464             delete depositAmounts[assetFeedNumber[i]][
465                 assetData[assetFeedNumber[i]].players[k]
466             ];
467         }
468     }
```

At this point, due to the `delete` operation, the array is empty. Therefore, the subsequent iteration and deletion operation is ineffective.

The `player` array should be iterated and deleted first, followed by the deletion of `assetData[assetFeedNumber[i]]`.

[XYRO team, 02/11/2025]:

The project team resolved this issue by implementing the recommendation suggested. Changes have been reflected in the commit hash: <https://github.com/xyro-io/smart-contracts/tree/d8bc6dcccdd3780cff138f57ad8f32cdf076f99d/contracts>

CON-01 FinalizeGame() NOT ACCOUNTING FOR topIndex POINTING TO ASSET FEED NUMBER INDEX WITH NO BETS

Category	Severity	Location	Status
Design Issue	● Medium	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 404~408; Treasury.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 607~618	● Resolved

Description

In `finalizeGame()`, the asset feed number with the highest `topDiff` is selected, and rewards are distributed to all players who bet on that asset feed number. However, the function does not account for the case where no bets are placed in asset feed number with `topDiff`. This results in a revert when calculating the rate due to

`assetData[assetFeedNumber[topIndex]].totalDeposits` being 0:

Race.sol

```
404         uint256 finalRate = ITreasury(treasury).calculateRate(  
405             assetData[assetFeedNumber[topIndex]].totalDeposits,  
406             totalLostRakeback,  
407             currentGameId  
408         );
```

Treasury.sol

```
607     function calculateRate(  
608         uint256 wonTeamTotal,  
609         uint256 lostTeamRakeback,  
610         bytes32 gameId  
611     ) public onlyRole(DISTRIBUTOR_ROLE) returns (uint256) {  
612         if (lostTeamRakeback != 0) {  
613             locked[gameId] -= lostTeamRakeback;  
614         }  
615         return  
616             ((locked[gameId] - wonTeamTotal) * RATE_PRECISION_AMPLIFIER) /  
617             wonTeamTotal;  
618     }
```

In `calculateRate`, a revert occurs due to division by zero.

Recommendation

There are two possible fixes for this issue:

1. When the asset feed number with the highest `topDiff` has no bets, refund all funds to the players.
2. Only select the asset feed number with the highest `topDiff` among the asset feed numbers that have bets.

■ Alleviation

[XYRO team, 02/09/2025]:

The project team resolved this issue by implementing the recommendation suggested. Changes have been reflected in the commit hash: <https://github.com/xyro-io/smart-contracts/tree/713050f0bdf4660b07c76e5625eb3350a014978f/contracts>

RAC-05 | NO CAP ON FEES

Category	Severity	Location	Status
Logical Issue	● Medium	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 544	● Resolved

Description

The `fee` variables in the contract have no set limits, allowing fees to potentially reach 100%. If this occurs, users would receive no tokens from the contract, resulting in a complete loss of their investment.

Recommendation

We recommend setting a reasonable cap on fees and providing adequate disclosure to the community.

Alleviation

[XYRO team, 02/07/2025]:

The project team resolved this issue by implementing the recommendation suggested. Changes have been reflected in the commit hash: <https://github.com/xyro-io/smart-contracts/tree/713050f0bdf4660b07c76e5625eb3350a014978f/contracts>

RAC-06 | SHOULD NOT `closeGame()` WHEN `finalPricesDiff[i]` EQUALS TO `topDiff` IN THE LOOP

Category	Severity	Location	Status
Logical Issue	● Medium	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 381~387	● Resolved

Description

In `finalizeGame()`, if the condition `topDiff == finalPricesDiff[i]` is encountered during the loop, the game is invalidated and `closeGame()` is called:

Race.sol

```
381         } else if (finalPricesDiff[i] == topDiff) {
382             emit RaceDraw(currentGameId);
383             closeGame();
384             return;
385         }
```

The design here is that if the two highest diffs among all assets are equal, the game should be invalidated. However, in the loop, `topDiff` is just an intermediate result and cannot represent the final `topDiff`. As a result, the game might be invalidated and closed under unexpected circumstances.

Scenario

For example, suppose there are 5 assets with final price changes of 10%, 10%, 10%, 10%, and 20%, with asset feed number of 20% change being at the end of the `assetFeedNumber` array. When `finalizeGame()` iterates through the earlier elements of the array, it satisfies the condition `topDiff == finalPricesDiff[i]` (i.e., `10% == 10%`). Therefore, the game will close, and all bets will be refunded to the players. This does not align with the intended game design.

Recommendation

It is recommended to maintain an array related to `topDiff` in `finalizeGame()` similar to the `Bullseye` contract, and then compare the top two values at the end to determine whether the highest two asset `topDiff` values are equal.

Alleviation

[XYRO team, 02/07/2025]:

Fixed this issue, please check commit hash:b4007e42cdd8af35fe144d4cb42c9eb4b29bc4fa

[CertiK, 02/10/2025]:

Thank you for your update and response. However, in the latest modification, `topDiff` is only calculated for assets with player bets, while `counter` is computed by iterating through all assets:

```
401     uint256 counter;
402     for (uint i; i < unverifiedReports.length; i++) {
403         if (finalPricesDiff[i] == topDiff) {
404             counter++;
405             if (counter == 2) {
406                 emit RaceDraw(currentGameId);
407                 closeGame();
408                 return;
409             }
410         }
411     }
```

This may lead to the following issue: Among all assets with player bets, only one asset has a `finalPricesDiff` equal to `topDiff`, while another asset without any player bets also has a `finalPricesDiff` equal to `topDiff`. In this case, even though there is only one winning asset, `closeGame()` would still be triggered.

Therefore, we recommend calculating `counter` only for assets where `assetData[assetFeedNumber[i]].players.length != 0`.

[XYRO team, 02/11/2025]:

The project team resolved this issue by implementing the recommendation suggested. Changes have been reflected in the commit hash: <https://github.com/xyro-io/smart-contracts/tree/d8bc6dccded3780cff138f57ad8f32cdf076f99d/contracts>

RAC-07 | MISSING CHECK IN `setAssetAmount()`

Category	Severity	Location	Status
Coding Issue	● Minor	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 522~523	● Resolved

Description

In `setAssetAmount()`, there is no check to ensure that `minAssetAmount` is less than or equal to `maxAssetAmount`:

Race.sol

```
518     function setAssetAmount(  
519         uint256 newMinAssetAmount,  
520         uint256 newMaxAssetAmount  
521     ) public onlyRole(DEFAULT_ADMIN_ROLE) {  
522         minAssetAmount = newMinAssetAmount;  
523         maxAssetAmount = newMaxAssetAmount;  
524         emit NewAssetCap(newMinAssetAmount, newMaxAssetAmount);  
525     }
```

Recommendation

It is recommended to add the relevant check.

Alleviation

[XYRO team, 02/07/2025]:

The project team resolved this issue by implementing the recommendation suggested. Changes have been reflected in the commit hash: <https://github.com/xyro-io/smart-contracts/tree/713050f0bdf4660b07c76e5625eb3350a014978f/contracts>

RAC-01 | CONFIRMATION ON RACE GAME

Category	Severity	Location	Status
Design Issue	● Informational	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 246~249, 374~377	● Acknowledged

Description

Regarding the Race game, we have the following questions:

1. The game design allows the same address to choose different asset feed numbers to play. Please confirm if this is as expected.
2. The `topDiff` calculated in `finalizeGame()` represents the largest asset price increase, not the largest asset price fluctuation. Therefore, the player who bets on the asset with the highest price increase will win and receive the prize. Please confirm if this aligns with the expected behavior.
3. If the probability of each asset winning in the game is equal, we have found a betting strategy that can yield a profit based on mathematical expectation. When the first player in each round of the game places a large bet, an attacker can place `MaxPlayer-1` bets, effectively reaching the `maxPlayer` limit and preventing other players from participating, while still being able to profit based on probability. We would like to confirm whether this is the expected behavior.

Scenario

For the third point in the discussion, we present the following scenario:

Consider the following scenario: In a round of the game, there are 5 assets, each with an equal chance of winning, and the game charges a 10% game fee. The `maxPlayer` is 100, the `minDepositAmount` is 1 USDT, and none of the participating players have `rakeBack`. The first player places a 1000 USDT bet on asset 1. Then, an arbitrage user uses different addresses to place a total of 99 bets of 1 USDT each on assets other than asset 1. After the game ends, if asset 1 wins, the arbitrageur loses 99 USDT. If any asset other than asset 1 wins, the arbitrageur will receive $1000 * 90\% = 900$ USDT.

Therefore, the mathematical expectation of the arbitrageur's profit is: $\frac{4}{5} * 900 - \frac{1}{5} * 99 = 700.2$ USDT

Simply find a higher bet amount, and when the first player's bet exceeds that amount, the operation can be carried out.

Recommendation

We would like to confirm whether this is the expected behavior.

Alleviation

[XYRO team, 02/10/2025]:

Everything works as intended, for the last questing we were analyzing player's behaviour and they never make big bets, and our UI allow to bet 20\$ at max, but anyway players tend to make smaller bets, so the last one seems impossible.

RAC-08 | SOLIDITY VERSION 0.8.23 WON'T WORK FOR ALL CHAINS DUE TO MCOPY

Category	Severity	Location	Status
Design Issue	● Informational	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 2	● Resolved

Description

Since Solidity Release 0.8.23, `MCOPY` opcode is introduced. However, this may not be compatible with all chains and L2s. As a result, the compatibility of the code could be affected.

Recommendation

To mitigate this issue, it is recommended to ensure the target chains support `MCOPY` or use an earlier solidity version.

Alleviation

[XYRO team, 02/07/2025]:

We will be aware of this issue if we consider deploying on other chains. For now Arbitrum is enough and it supports this opcode.

RAC-09 | EVENT NOT INDEXED

Category	Severity	Location	Status
Design Issue	● Informational	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 9~46	● Acknowledged

Description

If an event is not indexed in a smart contract, it means that the event's parameters are not tagged with the `indexed` keyword. This has implications for how the event data can be searched and filtered when looking through blockchain logs.

Without indexing, the event will still emit the data as part of the transaction log, but users won't be able to query for these events using the parameters. They'll have to retrieve the entire set of logs and manually sift through them to find events with the specific data. This can be less efficient and more time-consuming, especially on a blockchain with a high volume of transactions and events.

Recommendation

To mitigate this issue, it is recommended to index the most relevant parameters in the event to be defined.

Alleviation

[XYRO team, 02/07/2025]:

Issue acknowledged. I won't make any changes for the current version.

RAC-10 | SOLIDITY VERSION 0.8.20 MAY NOT WORK ON OTHER CHAINS DUE TO `PUSH0`

Category	Severity	Location	Status
Logical Issue	● Informational	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 2	● Acknowledged

Description

The compiler for Solidity 0.8.20 switches the default target EVM version to Shanghai, which includes the new `PUSH0` op code. This op code may not yet be implemented on all L2s, so deployment on these chains will fail. To work around this issue, use an earlier EVM version

Recommendation

It's recommended to pay attention to the EVM compiler version when using 0.8.20 solidity version in your contract.

Alleviation

[XYRO team, 02/07/2025]:

Issue acknowledged. I won't make any changes for the current version.

RAC-11 | MISSING CHECK FOR `minDepositAmount` IN `startGame()`

Category	Severity	Location	Status
Coding Issue	● Informational	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 91	● Resolved

Description

In `startGame()`, there was no check to verify if the `minDepositAmount` is greater than or equal to the corresponding token's `minDepositAmount` in the treasury. When the set value is too low, users may pass the `minDepositAmount` check in the game contract during play, but fail the `minDepositAmount` check in the treasury during `depositAndLock()`, `lock()`, and `depositAndLockWithPermit()`, resulting in a revert.

Recommendation

In `startGame()`, check if the value of `minDepositAmount` is greater than or equal to the corresponding token's `minDepositAmount` in the treasury.

Alleviation

[XYRO team, 02/07/2025]:

The project team resolved this issue by implementing the recommendation suggested. Changes have been reflected in the commit hash: <https://github.com/xyro-io/smart-contracts/tree/713050f0bdf4660b07c76e5625eb3350a014978f/contracts>

RAC-12 | POTENTIAL INCONSISTENT TIME RANGE FOR PRICE COMPARISON IN GAME FINALIZATION

Category	Severity	Location	Status
Design Issue	● Informational	Race.sol (1073bceba6f075d94e32883d4ac38127da2cd811): 324, 369	● Partially Resolved

Description

In the `finalizeGame` function, the contract currently checks if `priceTimestamp` is less than `endTime + 1 minutes`. This approach presents a risk due to potential significant fluctuations in token price within the 1-minute window, which could lead to biased game results. This logic does not ensure a consistent time range for price comparison, potentially affecting the fairness of the game. For instance, tokenA has a time range of 90 seconds while tokenB has a time range of 60 seconds, which can create an unfair advantage.

```
require(  
    priceTimestamp - game.endTime <= 1 minutes,  
    "Old chainlink report"  
);
```

Recommendation

We recommend adjusting the logic to ensure that the price comparison occurs within the same start and end time range.

Alleviation

[XYRO team, 02/11/2025]:

We lowered the time gap to 10 seconds. Can't get rid of the time range totally because we need to ensure that our will call these functions successfully. Please check this commit hash: 35b382cf4f7f9d12a61871f490716dd4b9b06464.

[CertiK, 02/12/2025]:

The current implementation reduces the risk of this issue. However, since the time check has been reduced from 1 minute to 10 seconds, the contract now has a higher dependency on the Chainlink data. If Chainlink returns an inaccurate price, it could potentially lead to a denial-of-service (DOS) situation.

FORMAL VERIFICATION | XYRO - AUDIT - RACE

Formal guarantees about the behavior of smart contracts can be obtained by reasoning about properties relating to the entire contract (e.g. contract invariants) or to specific functions of the contract. Once such properties are proven to be valid, they guarantee that the contract behaves as specified by the property. As part of this audit, we applied formal verification to prove that important functions in the smart contracts adhere to their expected behaviors.

Considered Functions And Scope

In the following, we provide a description of the properties that have been used in this audit. They are grouped according to the type of contract they apply to.

Verification of contracts derived from AccessControl v4.4

We verified properties of the public interface of contracts that provide an AccessControl-v4.4 compatible API. This involves:

- The `hasRole` function, which returns `true` if an account has been granted a specific `role`.
- The `getRoleAdmin` function, which returns the admin role that controls a specific `role`.
- The `grantRole` and `revokeRole` functions, which are used for granting a `role` to an account and revoking a `role` from an `account`, respectively.
- The `renounceRole` function, which allows the calling account to revoke a `role` from itself.

The properties that were considered within the scope of this audit are as follows:

Property Name	Title
accesscontrol-getroleadmin-succeed-always	<code>getRoleAdmin</code> Function Always Succeeds
accesscontrol-renouncerole-revert-not-sender	<code>renounceRole</code> Reverts When Caller Is Not the Confirmation Address
accesscontrol-getroleadmin-change-state	<code>getRoleAdmin</code> Function Does Not Change State
accesscontrol-renouncerole-succeed-role-renouncing	<code>renounceRole</code> Successfully Renounces Role
accesscontrol-hasrole-change-state	<code>hasRole</code> Function Does Not Change State
accesscontrol-default-admin-role	AccessControl Default Admin Role Invariance
accesscontrol-revokerole-correct-role-revoking	<code>revokeRole</code> Correctly Revokes Role
accesscontrol-grantrole-correct-role-granting	<code>grantRole</code> Correctly Grants Role
accesscontrol-hasrole-succeed-always	<code>hasRole</code> Function Always Succeeds

Verification Results

For the following contracts, formal verification established that each of the properties that were in scope of this audit (see scope) are valid:

Detailed Results For Contract Race (contracts/Race.sol) In Commit 713050f0bdf4660b07c76e5625eb3350a014978f

Verification of contracts derived from AccessControl v4.4

Detailed Results for Function `getRoleAdmin`

Property Name	Final Result	Remarks
accesscontrol-getroleadmin-succeed-always	● True	
accesscontrol-getroleadmin-change-state	● True	

Detailed Results for Function `renounceRole`

Property Name	Final Result	Remarks
accesscontrol-renouncerole-revert-not-sender	● True	
accesscontrol-renouncerole-succeed-role-renouncing	● True	

Detailed Results for Function `hasRole`

Property Name	Final Result	Remarks
accesscontrol-hasrole-change-state	● True	
accesscontrol-hasrole-succeed-always	● True	

Detailed Results for Function `DEFAULT_ADMIN_ROLE`

Property Name	Final Result	Remarks
accesscontrol-default-admin-role	● True	

Detailed Results for Function `revokeRole`

Property Name	Final Result	Remarks
accesscontrol-revokerole-correct-role-revoking	● True	

Detailed Results for Function `grantRole`

Property Name	Final Result	Remarks
accesscontrol-grantrole-correct-role-granting	● True	

Detailed Results For Contract Race (contracts/Race.sol) In Commit 1073bceba6f075d94e32883d4ac38127da2cd811**Verification of contracts derived from AccessControl v4.4**Detailed Results for Function `getRoleAdmin`

Property Name	Final Result	Remarks
accesscontrol-getroleadmin-succeed-always	● True	
accesscontrol-getroleadmin-change-state	● True	

Detailed Results for Function `hasRole`

Property Name	Final Result	Remarks
accesscontrol-hasrole-succeed-always	● True	
accesscontrol-hasrole-change-state	● True	

Detailed Results for Function `renounceRole`

Property Name	Final Result	Remarks
accesscontrol-renouncerole-succeed-role-renouncing	● True	
accesscontrol-renouncerole-revert-not-sender	● True	

Detailed Results for Function `grantRole`

Property Name	Final Result	Remarks
accesscontrol-grantrole-correct-role-granting	● True	

Detailed Results for Function `revokeRole`

Property Name	Final Result	Remarks
accesscontrol-revokerole-correct-role-revoking	● True	

Detailed Results for Function `DEFAULT_ADMIN_ROLE`

Property Name	Final Result	Remarks
accesscontrol-default-admin-role	● True	

APPENDIX | XYRO - AUDIT - RACE

Finding Categories

Categories	Description
Coding Issue	Coding Issue findings are about general code quality including, but not limited to, coding mistakes, compile errors, and performance issues.
Denial of Service	Denial of Service findings indicate that an attacker may prevent the program from operating correctly or responding to legitimate requests.
Logical Issue	Logical Issue findings indicate general implementation issues related to the program logic.
Centralization	Centralization findings detail the design choices of designating privileged roles or other centralized controls over the code.
Design Issue	Design Issue findings indicate general issues at the design level beyond program logic that are not covered by other finding categories.

Checksum Calculation Method

The "Checksum" field in the "Audit Scope" section is calculated as the SHA-256 (Secure Hash Algorithm 2 with digest size of 256 bits) digest of the content of each file hosted in the listed source repository under the specified commit.

The result is hexadecimal encoded and is the same as the output of the Linux "sha256sum" command against the target file.

Details on Formal Verification

Some Solidity smart contracts from this project have been formally verified. Each such contract was compiled into a mathematical model that reflects all its possible behaviors with respect to the property. The model takes into account the semantics of the Solidity instructions found in the contract. All verification results that we report are based on that model.

The following assumptions and simplifications apply to our model:

- Certain low-level calls and inline assembly are not supported and may lead to a contract not being formally verified.
- We model the semantics of the Solidity source code and not the semantics of the EVM bytecode in a compiled contract.

Formalism for property specifications

All properties are expressed in a behavioral interface specification language that CertiK has developed for Solidity, which allows us to specify the behavior of each function in terms of the contract state and its parameters and return values, as well as contract properties that are maintained by every observable state transition. Observable state transitions occur when the

contract's external interface is invoked and the invocation does not revert, and when the contract's Ether balance is changed by the EVM due to another contract's "self-destruct" invocation. The specification language has the usual Boolean connectives, as well as the operator `\old` (used to denote the state of a variable before a state transition), and several types of specification clause:

Apart from the Boolean connectives and the modal operators "always" (written `[]`) and "eventually" (written `<>`), we use the following predicates to reason about the validity of atomic propositions. They are evaluated on the contract's state whenever a discrete time step occurs:

- `requires [cond]` - the condition `cond`, which refers to a function's parameters, return values, and contract state variables, must hold when a function is invoked in order for it to exhibit a specified behavior.
- `ensures [cond]` - the condition `cond`, which refers to a function's parameters, return values, and both `\old` and current contract state variables, is guaranteed to hold when a function returns if the corresponding requires condition held when it was invoked.
- `invariant [cond]` - the condition `cond`, which refers only to contract state variables, is guaranteed to hold at every observable contract state.
- `constraint [cond]` - the condition `cond`, which refers to both `\old` and current contract state variables, is guaranteed to hold at every observable contract state except for the initial state after construction (because there is no previous state); constraints are used to restrict how contract state can change over time.

Description of the Analyzed AccessControl-v4.4 Properties

Properties related to function `getRoleAdmin`

accesscontrol-getroleadmin-change-state

The `getRoleAdmin` function must not change any state variables.

Specification:

```
assignable \nothing;
```

accesscontrol-getroleadmin-succeed-always

The `getRoleAdmin` function must always succeed, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `renounceRole`

accesscontrol-renouncerole-revert-not-sender

The `renounceRole` function must revert if the caller is not the same as `account`.

Specification:

```
reverts_when account != msg.sender;
```

accesscontrol-renouncerole-succeed-role-renouncing

After execution, `renounceRole` must ensure the caller no longer has the renounced role.

Specification:

```
ensures !hasRole(role, account);
```

Properties related to function `hasRole`

accesscontrol-hasrole-change-state

The `hasRole` function must not change any state variables.

Specification:

```
assignable \nothing;
```

accesscontrol-hasrole-succeed-always

The `hasRole` function must always succeed, assuming that its execution does not run out of gas.

Specification:

```
reverts_only_when false;
```

Properties related to function `DEFAULT_ADMIN_ROLE`

accesscontrol-default-admin-role

The default admin role must be invariant, ensuring consistent access control management.

Specification:

```
invariant DEFAULT_ADMIN_ROLE() == 0x00;
```

Properties related to function `revokeRole`

accesscontrol-revokerole-correct-role-revoking

After execution, `revokeRole` must ensure the specified account no longer has the revoked role.

Specification:

```
ensures !hasRole(role, account);
```

Properties related to function `grantRole`

accesscontrol-grantrole-correct-role-granting

After execution, `grantRole` must ensure the specified account has the granted role.

Specification:

```
ensures hasRole(role, account);
```

DISCLAIMER | CERTIK

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to you ("Customer" or the "Company") in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes, nor may copies be delivered to any other person other than the Company, without CertiK's prior written consent in each instance.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts CertiK to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. CertiK's position is that each company and individual are responsible for their own due diligence and continuous security. CertiK's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by CertiK is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

ALL SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF ARE PROVIDED "AS IS" AND "AS AVAILABLE" AND WITH ALL FAULTS AND DEFECTS WITHOUT WARRANTY OF ANY KIND. TO THE MAXIMUM EXTENT PERMITTED UNDER APPLICABLE LAW, CERTIK HEREBY DISCLAIMS ALL WARRANTIES, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHERWISE WITH RESPECT TO THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS. WITHOUT LIMITING THE FOREGOING, CERTIK SPECIFICALLY DISCLAIMS ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, TITLE AND NON-INFRINGEMENT, AND ALL WARRANTIES ARISING FROM COURSE OF DEALING, USAGE, OR TRADE PRACTICE. WITHOUT LIMITING THE FOREGOING, CERTIK MAKES NO WARRANTY OF ANY KIND THAT THE SERVICES, THE LABELS, THE ASSESSMENT REPORT, WORK PRODUCT, OR OTHER MATERIALS, OR ANY PRODUCTS OR RESULTS OF THE USE THEREOF, WILL MEET CUSTOMER'S OR ANY OTHER PERSON'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULT, BE COMPATIBLE OR WORK WITH ANY SOFTWARE, SYSTEM, OR OTHER SERVICES, OR BE SECURE, ACCURATE, COMPLETE, FREE OF HARMFUL CODE, OR ERROR-FREE. WITHOUT LIMITATION TO THE FOREGOING, CERTIK PROVIDES NO WARRANTY OR

UNDERTAKING, AND MAKES NO REPRESENTATION OF ANY KIND THAT THE SERVICE WILL MEET CUSTOMER'S REQUIREMENTS, ACHIEVE ANY INTENDED RESULTS, BE COMPATIBLE OR WORK WITH ANY OTHER SOFTWARE, APPLICATIONS, SYSTEMS OR SERVICES, OPERATE WITHOUT INTERRUPTION, MEET ANY PERFORMANCE OR RELIABILITY STANDARDS OR BE ERROR FREE OR THAT ANY ERRORS OR DEFECTS CAN OR WILL BE CORRECTED.

WITHOUT LIMITING THE FOREGOING, NEITHER CERTIK NOR ANY OF CERTIK'S AGENTS MAKES ANY REPRESENTATION OR WARRANTY OF ANY KIND, EXPRESS OR IMPLIED AS TO THE ACCURACY, RELIABILITY, OR CURRENCY OF ANY INFORMATION OR CONTENT PROVIDED THROUGH THE SERVICE. CERTIK WILL ASSUME NO LIABILITY OR RESPONSIBILITY FOR (I) ANY ERRORS, MISTAKES, OR INACCURACIES OF CONTENT AND MATERIALS OR FOR ANY LOSS OR DAMAGE OF ANY KIND INCURRED AS A RESULT OF THE USE OF ANY CONTENT, OR (II) ANY PERSONAL INJURY OR PROPERTY DAMAGE, OF ANY NATURE WHATSOEVER, RESULTING FROM CUSTOMER'S ACCESS TO OR USE OF THE SERVICES, ASSESSMENT REPORT, OR OTHER MATERIALS.

ALL THIRD-PARTY MATERIALS ARE PROVIDED "AS IS" AND ANY REPRESENTATION OR WARRANTY OF OR CONCERNING ANY THIRD-PARTY MATERIALS IS STRICTLY BETWEEN CUSTOMER AND THE THIRD-PARTY OWNER OR DISTRIBUTOR OF THE THIRD-PARTY MATERIALS.

THE SERVICES, ASSESSMENT REPORT, AND ANY OTHER MATERIALS HEREUNDER ARE SOLELY PROVIDED TO CUSTOMER AND MAY NOT BE RELIED ON BY ANY OTHER PERSON OR FOR ANY PURPOSE NOT SPECIFICALLY IDENTIFIED IN THIS AGREEMENT, NOR MAY COPIES BE DELIVERED TO, ANY OTHER PERSON WITHOUT CERTIK'S PRIOR WRITTEN CONSENT IN EACH INSTANCE.

NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH SERVICES, ASSESSMENT REPORT, AND ANY ACCOMPANYING MATERIALS.

THE REPRESENTATIONS AND WARRANTIES OF CERTIK CONTAINED IN THIS AGREEMENT ARE SOLELY FOR THE BENEFIT OF CUSTOMER. ACCORDINGLY, NO THIRD PARTY OR ANYONE ACTING ON BEHALF OF ANY THEREOF, SHALL BE A THIRD PARTY OR OTHER BENEFICIARY OF SUCH REPRESENTATIONS AND WARRANTIES AND NO SUCH THIRD PARTY SHALL HAVE ANY RIGHTS OF CONTRIBUTION AGAINST CERTIK WITH RESPECT TO SUCH REPRESENTATIONS OR WARRANTIES OR ANY MATTER SUBJECT TO OR RESULTING IN INDEMNIFICATION UNDER THIS AGREEMENT OR OTHERWISE.

FOR AVOIDANCE OF DOUBT, THE SERVICES, INCLUDING ANY ASSOCIATED ASSESSMENT REPORTS OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.

Elevating Your Entire **Web3** Journey

Founded in 2017 by leading academics in the field of Computer Science from both Yale and Columbia University, CertiK is a leading blockchain security company that serves to verify the security and correctness of smart contracts and blockchain-based protocols. Through the utilization of our world-class technical expertise, alongside our proprietary, innovative tech, we're able to support the success of our clients with best-in-class security, all whilst realizing our overarching vision; provable trust for all throughout all facets of blockchain.

